

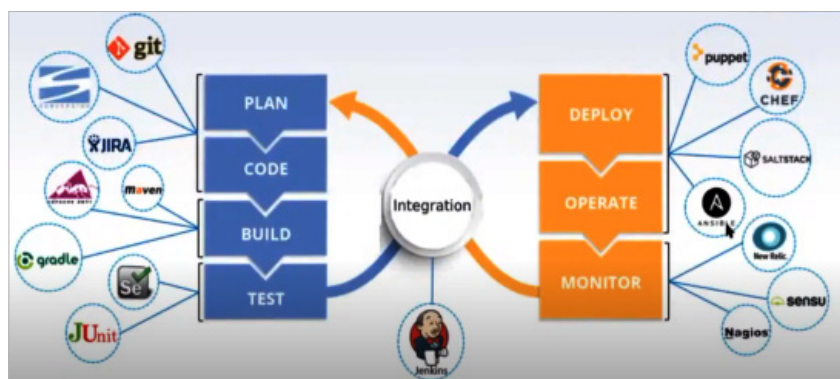


Figure 2: Integration of various DevOps tools with DevSecOps tools

Tools available at your disposal

Some of the tools that are already being put to use in the DevOps toolchain can be used to manage DevSecOps too. No new tools are needed for DevSecOps. Tools such as Jira, Git, Gradle, and Selenium are a few that can be used for planning, coding, building and testing software. They are used in DevOps but are compliant with DevSecOps as well — it is just a matter of how they are implemented. Tools such as Puppet, Chef, and Sensu can be used for deployment, operation and monitoring (refer to Figure 2).

figure out the issues right from the word go rather than finding them at the end. It allows them to respond to the changes rapidly when there are a lot of new laws that have to be abided by such as the General Data Protection Regulation (GDPR). When the security team is involved from the beginning, it helps them to deal with issues more swiftly. More importantly, it helps to create more automated builds, and more QA testing can be done.

Automating the entire process in DevOps is a very crucial aspect, and is one of the core pillars on which it stands today. The more things can be structured and automated the better the software is, as it tends to be more scalable that way. Automation helps in maintaining better communication between the security and development teams, which is something DevOps has always advocated.

Figure 1 depicts the workflow that DevOps usually advocates, on top of which the layers of security are supposed to be added at the planning stage. This is the stage at which the security team is already thinking about

what kind of threat model policies they want to build in, and the issues that they could possibly encounter when deploying the final product.

Once the coding pipeline where code is being generated is reached, the static analysis and code reviews are to be done dynamically. When it gets into testing, the security team must make sure that the code is hardened against the known vulnerabilities. This will significantly lower the risk of attacks. Some kind of pen testing can be done while making sure that the security planning happens at the planning stage itself and not at the test stage.

Once the code is ready to be deployed, it is essential to continuously monitor and assess it for any kind of threats and vulnerabilities so that the application is not impacted. It must also meet all necessary compliances.

The benefits of using security as code

As much as the automation of the entire workflow is important, there is an extent to which it can be done using tools. Managing a single server manually is

not a big deal, but to manage tens of thousands of servers together every day can be super challenging. A code can be put in place in the form of an application, which can very easily manage these servers on its own. It can be instructed to do so according to the required security protocol. Let us look at some of the benefits of using security as code.

Collaboration: A code is an unambiguous common language that can be understood by literally everyone, even if a person doesn't have any prior understanding of it. Anyone can learn to code and it will perform the same way wherever it is implemented. This makes it significantly easier to work with people from all walks of life.

Scalability: It is highly scalable. Irrespective of the number of servers or nodes being managed, a piece of code is going to perform the same way wherever it has been deployed. This leaves almost no room for error and minimises the risk of malfunctions.

Visibility: Since the code is always visible, it can be edited anytime it is necessary to do so. Whenever there is a bug that might cause any kind of hindrance, it can be rectified very easily without any significant damage. It can be altered at will at any given point of time.

It's a no-brainer that security shall always be the utmost priority for anything that you do on the Web, and anything that comprises system hardware and software. Weak security can lead to irreversible losses. Hence, it becomes imperative for developers to implement foolproof security using evolving philosophies like DevSecOps. **END** 🐧

By: Jonathan Pereira

The author is DevRel at Polygon. This article is based on a talk he gave at OSI 2021, a virtual event.

*Transcribed and curated by
Laveesh Kocher*